

第一章 术语

一、名词解释

1. 操作系统

指控制和管理计算机中各种硬件和软件资源有效
组织多道程序并发执行的系统软件

4. ^{系统}实时指能及时响应外部事件的请求，在规定的

时间内完成对该事件的处理，并控制所有实时任务协调有效地运行。

5. 互斥共享：多个进程系统中某些资源可提供给多个进程使用，但规定一段时间内只允许一个进程访问该资源。(进程)

二、硬件系统，软件系统

3. 硬件资源，软件资源

5. 共享性 互斥性

第二章

3. 每进程中访问临界资源的用一段代码称为临界区。

No.
Date

4. 进程同步: 进程之间必须按照一定的先后顺序执行

二. 动态性, ^{封闭性} 共享性

5. 动态, 静态

6. ~~进程的阻塞~~ 同步

8. 互斥通信, 同步消息 ^{队列} ^{共享内存}

9. ~~执行~~ 就绪 ^{阻塞}

2.
S₁ 无前驱为起始进程, S₂ 前驱为 S₁, S₃ 前驱为 S₁,
S₄ 前驱为 S₂, S₅ 前驱为 S₂, S₃, S₆ 前驱为 S₃, S₇ 前驱为
S₄, S₅, S₆

~~semaphore~~ semaphore a=0, b=0, c=0, d=0, e=0, f=0, g=0, h=0, i=0;
void S₁() {
 signal(a);
 signal(b); }
void S₂() {
 wait(a);
 signal(c);
 signal(d); }
void S₃() {
 wait(c);
 signal(e);
 signal(f); }
void S₄() {
 wait(e);
 signal(g); }
void S₅() {
 wait(d);
 wait(e);
 signal(h); }
void S₆() {
 wait(f);
 signal(i); }
}

接 2.2

void Stc() {

wait(h);

wait(h);

wait(i);

}

int main() {

// 创建并启动所有线程

1. 本程序使用线程库实现父子进程通信

return 0;

}

3. #include

Semaphore mutex=1;

Semaphore empty=5;

Semaphore orange=0;

Semaphore apple=0;

void father() {

while(true) {

fruit = random(apple, orange);

wait(empty); wait(mutex);

if (fruit == orange) {

signal(orange); }

else { signal(apple); }

signal(mutex); }

void son() {

while(true) {

wait(orange);

wait(mutex);

signal(empty);

signal(mutex);

}

void daughter() {

while(true) {

wait(apple);

wait(mutex);

signal(empty);

signal(mutex);

}

}

4. 文件打印程序

3A-3A4 2034380p

Semaphore empty1=1; Semaphore full1=0; Semaphore empty2=1;

Semaphore full2=0;

void p1() {

while (true) {

wait(empty1);

signal(full1); }

void p2() { while (true) {

wait(full1);

wait(empty2);

signal(empty1);

signal(full2); }

void p3() {

while (true) {

wait(full2);

signal(empty2);

}

}

5. 理发店

Semaphore customers=0; Semaphore barber=0; Semaphore mutex=1;

Semaphore payment=0; Semaphore receipt=0;

int waiting=0;

const int N=5;

void barber-process() {

wait(true) {

wait(customers);

wait(mutex);

waiting--;

signal(barber);

signal(mutex);

wait(payment);

signal(receipt); }

void customer-process() {

wait(mutex);

if (waiting > N) {

signal(mutex);

return;

waiting++;

signal(customers);

signal(mutex);

wait(barber);

signal(payment);

wait(receipt);

}

第三章

孙浩阳 2023/11/14

一、

2. 操作系统的某种算法从就绪队列中选择一个进程并分配CPU资源给其执行的进程

3. 指从作业提交到系统开始,到作业完成所消耗的时间。

4. 导致任一进程相互竞争系统资源或进行通信的进程间的再次阻塞。

三、~~提交~~

2. 提交, 调度, 完成

6. 预防避免, 检测, 恢复

五、

1. (CFS)

A=0-3 B=3-9 C=9-13 D=13-18 E=18-20

完成时间 A=3, B=9, C=13, D=18, E=20

周转时间: A=3, B=7, C=9, D=12, E=12

带权周转: A=1, B=1.7, C=2.25, D=2.4, E=6

平均带权周转: $\frac{1+1.7+2.25+2.4+6}{5} = 2.59$

2-(SPTT)

非抢占

A=0-3, B=3-9, C=9-13, D=13-18, E=18-20

抢占SPT: A=0-3, B=3-5, C=5-11, D=11-15, E=15-20

完成时间 = A=3, E=5, B=11, C=15, D=20

加权平均 = A=1, E=1.5, B=1.5, C=2.75, D=2.8

平均 = $\frac{1+1.5+1.5+2.75+2.8}{5} = 1.95$

3. (HRRN): 完成时间: A=3, B=9, C=13, D=18, E=20

加权平均: A=1, B=1.7, C=2.25, D=2.4, E=2.6

平均 = $\frac{1+1.7+2.25+2.4+2.6}{5} = 2.59$

4. (RR, 时间片为1) 完成: A=19, B=16, C=20, D=20, E=20

加权平均: A=1.9, B=2.4, C=2.6, D=2.8, E=2.8

平均 = $\frac{1.9+2.4+2.6+2.8+2.8}{5} = 2.6$

② 1. 0ms 调度 A, 10ms 调度 B, 20ms 调度 A, 30ms 调度 C, 45ms 调度 B, 50ms 调度 A.

③ p0: 0032

p1: 100

p2: 1354

p3: 0332

p4: 0014

request(12, 20) = Need(12, 36) 满足

request(12, 20) ≤ available(16, 20) 满足

作调度分配 Available 变为 0400, Allocation 变为 2576

Need(p0) 变为 134

初始时 p0 的 Need 1750 小于 0400 不能调度

调度, 系统不安全, 不能调度 p2.

(3) 由于条件

满足, 满足

且系统不

安全, 可能

发生死

进程顺序为 p0 → p3 → p0 → p2

→ p4, 则安全